STUDY NOTES | OCTOBER 2026 EDITION

Cloud Computing and AWS Services

Extended study notes: cloud models, IaaS / PaaS / SaaS, deployment models, every core AWS service explained in depth, plus CloudWatch and SNS.

IAM S3 EC2 VPC RDS ELB Auto Scaling CloudWatch SNS

CREATED BY

Mohit Gadilohar

LINKEDIN

www.linkedin.com/in/mohit-gadilohar-devops

Companion to the guide AWS Cloud Fundamentals by Mohit Gadilohar (October 2026 edition).
Written for beginners. Limits, Regions and prices change, so verify figures on the official AWS pages.

Contents

What is inside these notes

01 What is cloud computing? . . . 3

02 Service models: IaaS, PaaS, SaaS . . . 4

03 Cloud deployment models . . . 5

04 AWS global infrastructure and IAM . . . 6

05 Amazon S3 . . . 10

06 Amazon EC2 . . . 14

07 Amazon VPC . . . 18

08 Amazon RDS . . . 22

09 Elastic Load Balancing . . . 26

10 EC2 Auto Scaling . . . 29

11 Amazon CloudWatch . . . 33

12 Amazon SNS . . . 36

13 Other related services . . . 39

14 Putting it together . . . 40

15 Best practices and cheat sheet . . . 41

16 Interview questions and answers . . . 42

01 What is cloud computing?

Renting IT resources over the internet and paying only for what you use

Cloud computing means using servers, storage, databases, networking and software that live in a provider's data centres and are delivered over the internet. Instead of buying hardware, you rent capacity and pay for what you consume, much like an electricity bill.

Before the cloud

A company bought servers, built a server room, hired staff to run it and guessed how much capacity it would need. Too little capacity meant crashes during busy periods. Too much meant money wasted on idle machines.

With the cloud

You create a server in minutes, use it as long as you need, then delete it. You can scale up for a sale and scale down afterwards.

Key characteristics

Characteristic	Meaning
On-demand self-service	You create and delete resources yourself, with no sales call.
Elasticity	Capacity grows and shrinks automatically or manually as demand changes.
Pay-as-you-go	Billing per second, hour, GB or request.
Broad network access	Resources are reached over the internet or private links.
Resource pooling	Many customers share a huge infrastructure, securely isolated from each other.
Measured service	Usage is metered, so you see exactly what you consume and pay for.
Global reach	Deploy close to your users in many Regions around the world.

Benefits and trade-offs

- **Benefits:** no upfront hardware cost, fast to start, scales easily, built-in resilience, global reach.
- **Trade-offs:** ongoing bills that can grow if unmanaged, dependence on the provider, and you must learn the shared responsibility model.

02 Service models: IaaS, PaaS, SaaS

How much the provider manages for you

The three service models describe how much of the technology stack you manage and how much the provider manages. Moving from IaaS to PaaS to SaaS, you get less control but less work.

Layer	On-premises	IaaS	PaaS	SaaS
Application	You	You	You	Provider
Data	You	You	You	Provider
Runtime / middleware	You	You	Provider	Provider
Operating system	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers, storage, network	You	Provider	Provider	Provider

IaaS: Infrastructure as a Service

You rent raw building blocks: virtual servers, storage and networks. You install the operating system, software and security patches yourself. This gives the most control and needs the most administration.

AWS examples: EC2, EBS, VPC, S3. Use it when you need custom software, special OS settings or a lift-and-shift migration of existing servers.

PaaS: Platform as a Service

The provider runs servers, OS and runtime, and you only deploy your code or data. Developers focus on the application instead of server upkeep.

AWS examples: Elastic Beanstalk, RDS (managed database), Lambda (serverless). Use it when you want to ship quickly without server administration.

SaaS: Software as a Service

A complete application delivered over the internet. You log in and use it, with nothing to install or patch.

Examples: Gmail, Microsoft 365, Salesforce, Zoom; on AWS, Amazon WorkMail and Amazon Chime.

Model	Pizza analogy	Typical user
On-premises	Make pizza at home: you buy and do everything	Legacy data centres
IaaS	Buy the base, add toppings, bake it yourself	System administrators, DevOps
PaaS	Order takeaway: you only eat	Application developers
SaaS	Dine at a restaurant	End users and business teams

NOTE Real services do not always fit one box. RDS is often called PaaS because AWS manages the OS and database engine, while S3 is usually described as storage-as-a-service. Treat the labels as a guide to how much you manage, not strict categories.

03 Cloud deployment models

Where the cloud runs and who can use it

Model	What it is	Best for	Cost / control
Public cloud	Owned by a provider (AWS, Azure, Google Cloud) and shared by many isolated customers. No upfront cost, near-unlimited scale.	Startups, web apps, most workloads	Lowest cost, moderate control
Private cloud	Used by one organization only, in its own data centre or hosted privately. Maximum control and compliance, but you maintain it.	Banks, government, defence	Highest cost, highest control
Hybrid cloud	Public and private combined and connected, for example regulated data on-premises and the website on AWS. AWS Direct Connect or VPN provide the link.	Gradual migration, regulated data	Medium cost, high control
Community cloud	Shared by several organizations with common needs, such as hospitals or universities with the same compliance rules.	Groups with shared rules	Shared cost
Multi-cloud	Using more than one provider, such as AWS plus Azure, to avoid lock-in or use the best service from each.	Large enterprises	Complex to manage

VPC IS NOT A PRIVATE CLOUD A VPC is an isolated network inside the public AWS cloud. It feels private, but it still runs on shared AWS infrastructure. AWS Outposts, or on-premises VMware and OpenStack, are closer to a true private cloud.

04 AWS global infrastructure and IAM

Where resources live and who may use them

Concept	Meaning	Analogy
Region	A geographic area with several data centre groups, such as Mumbai (ap-south-1) or N. Virginia (us-east-1). Data stays in the Region you choose.	A city
Availability Zone	One or more isolated data centres inside a Region with independent power and networking, linked by fast private networks.	A neighbourhood
Edge location	A smaller site that caches content close to users (used by CloudFront).	A local shop branch
Shared responsibility	AWS secures the cloud (hardware, facilities). You secure what you put in it (data, access, configuration).	Landlord locks the building, you lock your flat

IAM: Identity and Access Management

IAM decides who can sign in to your account and what they may do. It is free and global, and it is the first thing to set up properly.

BASICS: WHAT IS IAM AND WHY USE IT?

What is it?

IAM (Identity and Access Management) is the AWS service that controls who can sign in to your AWS account and what each person or application is allowed to do. Think of it as the ID-card and security-guard system of your account.

Why is it used?

- Without IAM everyone would share the single all-powerful root login, which is very risky.
- Each person and each application needs its own identity, so you can see who did what.
- Applications running on AWS need permissions without storing passwords in code.
- You want to give people only the access their job needs (least privilege).

Advantages

- Free to use, with no extra charge.
- Very fine-grained control: you can allow one action on one resource.
- Multi-factor authentication adds a second layer of protection.
- Temporary credentials through roles reduce the damage of a leak.
- Works with every AWS service and can connect to your company's sign-in (single sign-on).

Things to watch

- A mistake in a policy can either lock people out or open too much access.
- Root user and access keys must be protected. Never put them in code or share them.
- Permissions can become complicated in large accounts, so review them regularly.

Key terms in simple words

Term	Simple meaning	Everyday comparison
Root user	The account owner, who can do everything.	The building owner with the master key
IAM user	One person or application with its own login.	An employee ID card
Group	A set of users who share the same permissions.	A department such as Accounts
Role	Permissions that a person or service borrows for a short time.	A visitor pass that expires
Policy	A written rulebook (JSON) listing what is allowed or denied.	The list that says which doors a card opens
MFA	A second proof of identity besides the password.	Password plus a code on your phone
Least privilege	Give only the permissions that are really needed.	A cashier gets the till key, not the safe key

EXAMPLE A company has developers, testers and a website running on EC2. Developers get a group with limited access, the website's server gets a role that can read one S3 bucket, and only the owner uses the root user, protected with MFA.

Term	Meaning
Root user	The account owner. Enable MFA and use it only for rare account-level tasks.
IAM user	A person or application with its own credentials.
Group	A set of users sharing the same permissions, such as Developers.
Role	Temporary permissions that a service (for example EC2) or a person can assume, with no long-term password.
Policy	A JSON document listing allowed or denied actions on resources. Anything not explicitly allowed is denied.

Golden rules

- Turn on MFA for the root user, then stop using root.
- Give each person their own identity and never share credentials.
- Grant least privilege: only the permissions a task needs.
- Prefer roles over access keys for applications running on AWS.

Example policy

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Action": "s3:GetObject",
 "Resource": "arn:aws:s3:::my-unique-bucket-name/*"
 }]
}
```

IN DEPTH: IAM

How AWS decides who may do what

Authentication proves who you are (password plus MFA, access keys, or a role session). **Authorization** decides what you may do once you are known. IAM handles both, and every request to AWS is checked against policies before it is allowed.

How a request is evaluated

- Everything is denied by default (implicit deny).
- An applicable Allow in an identity policy or resource policy can grant access.
- An explicit Deny anywhere always beats any Allow.
- Guardrails (SCPs, permissions boundaries, session policies) set the maximum permissions. They never grant access on their own.

Policy types

Type	Attached to	Purpose
Identity-based policy	User, group or role	Says what that identity may do. The most common type.
Resource-based policy	A resource (S3 bucket, SQS queue, KMS key, role trust)	Says who may use the resource, including other accounts. Contains a Principal.
Permissions boundary	User or role	Caps the maximum permissions an identity can ever have.
Service control policy (SCP)	AWS Organizations account or OU	Caps what every identity in the account can do, even the root user of that member account.
Session policy	A temporary role session	Further narrows a session created with AssumeRole.
ACL	S3 objects and buckets (legacy)	Older access control. Disabled by default on new buckets.

Anatomy of a policy

Element	Meaning
Version	Policy language version. Always use 2012-10-17.
Statement	One or more permission rules.
Sid	Optional label for a statement.
Effect	Allow or Deny.
Principal	Who the rule applies to. Used only in resource-based and trust policies.
Action / NotAction	The API calls covered, such as s3:GetObject. Wildcards like s3:* are allowed.
Resource	The ARNs the rule applies to. An ARN looks like arn:partition:service:region:account-id:resource.
Condition	Extra tests such as source IP, MFA present, time, or resource tags.

Example: read-only S3 access, only with MFA and from an office network

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Sid": "ReadOnlyFromOfficeWithMFA",
 "Effect": "Allow",
 "Action": ["s3:GetObject", "s3:ListBucket"],
 "Resource": ["arn:aws:s3:::my-unique-bucket-name",
 "arn:aws:s3:::my-unique-bucket-name/*"],
 "Condition": {
 "Bool": { "aws:MultiFactorAuthPresent": "true" },
 "IpAddress": { "aws:SourceIp": "203.0.113.0/24" }
 }
 }]
}
```

Note the two Resource entries: s3:ListBucket applies to the bucket itself, while s3:GetObject applies to the objects inside it (the /* form). Forgetting one of them is a classic cause of Access Denied errors.

Roles and temporary credentials

A role has two parts: a **trust policy** (who may assume the role) and one or more **permissions policies** (what the role may do). When something assumes a role, AWS Security Token Service (STS) hands it temporary credentials made of an access key, a secret key and a session token. They expire automatically (by default after one hour, configurable from 15 minutes up to 12 hours depending on the role), so a leak is far less dangerous than a leaked long-term key.

```
# Trust policy that lets EC2 assume the role
{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Principal": { "Service": "ec2.amazonaws.com" },
 "Action": "sts:AssumeRole"
 }]
}
```

Role use	Who assumes it	Why
EC2 instance profile	An EC2 instance	Application reads S3 or DynamoDB with no stored keys.
Lambda execution role	A Lambda function	Gives the function permissions to call other services.
Cross-account role	A user or role in another account	Share access without sharing passwords.
Federated / IAM Identity Center	People signing in with company identity	Single sign-on with temporary access.
Service-linked role	An AWS service itself	Predefined by AWS so the service can manage resources for you.

Organizations, SCPs and IAM Identity Center

- AWS Organizations groups many accounts under one management account, with consolidated billing and SCP guardrails.
- IAM Identity Center gives people single sign-on across accounts using permission sets, so you create fewer IAM users.
- A common structure uses separate accounts for production, development and security tooling, so a mistake in one cannot touch the others.

Tools that keep IAM tidy

Tool	What it tells you
Credential report	Every user, whether MFA is on, and the age of passwords and access keys.
Access Advisor / last accessed	Which services a user or role has actually used, so you can remove unused permissions.
IAM Access Analyzer	Resources shared outside your account, and can generate least-privilege policies from CloudTrail activity.
Policy simulator	Tests whether a policy would allow or deny a given action before you deploy it.
CloudTrail	Audit record of every API call, including who made it.

Common mistakes

- Using Action "*" with Resource "*" because it is quick. It grants full administrator rights.
- Committing access keys to GitHub or baking them into an AMI.
- Sharing one IAM user between several people, so audit logs cannot tell them apart.
- Leaving old users and unused access keys active.
- Writing a trust policy that lets any principal in any account assume a powerful role.

```
aws iam create-role --role-name app-role --assume-role-policy-document file://trust.json
aws iam attach-role-policy --role-name app-role \
 --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
aws iam create-instance-profile --instance-profile-name app-profile
aws iam add-role-to-instance-profile --instance-profile-name app-profile --role-name app-role
aws iam generate-credential-report
```

05 Amazon S3

Object storage for any amount of data

BASICS: WHAT IS AMAZON S3 AND WHY USE IT?

What is it?

S3 (Simple Storage Service) is AWS's online storage for files. You upload any kind of file, it is stored safely, and you can fetch it from anywhere over the internet. It is not a drive attached to a computer: you use it through the console, the command line, or code. The two main ideas are the bucket (the container) and the object (the file inside it).

Why is it used?

- Backups and archives of important data.
- Images, videos and downloads for websites and apps.
- Hosting a static website (HTML, CSS, JavaScript).
- Storing logs and large data sets for analysis (data lakes).
- Sharing files safely with other people or systems.

Advantages

- Practically unlimited storage that grows automatically, with no disks to buy.
- Extremely durable: designed so that data is almost never lost (eleven nines).
- Pay only for what you store and use, with cheaper storage classes for rarely used data.
- Built-in security: encryption, access policies, Block Public Access.
- Versioning and lifecycle rules protect against deletes and control cost.
- Works with almost every other AWS service.

Things to watch

- S3 is not a file system or disk for a server. Use EBS or EFS for that.
- A public bucket can leak data, so keep Block Public Access on.
- You pay for data transfer out to the internet and for requests, not only for storage.
- Bucket names are global, so the name must be unique and cannot be renamed.

Key terms in simple words

Term	Simple meaning	Everyday comparison
Bucket	A container that holds your files. It is created in one Region and has a globally unique name.	A cabinet or top-level folder with a unique label
Object	One file plus its details (metadata). It can be up to 5 TB.	A document inside the cabinet
Key	The unique name or path of an object in the bucket, such as images/logo.png.	The file's address or label
Metadata	Information about the object, such as type, size and last modified date.	The details written on a document's cover
Storage class	The price and speed level you choose for an object.	Shop shelf (fast, costly) vs warehouse (slow, cheap)
Region	The geographic area where the bucket is stored.	The city where the cabinet is kept
Versioning	Keeps old copies when you overwrite or delete.	Track changes with older drafts saved
Presigned URL	A temporary link to a private object.	A guest pass for one room, valid for a few hours

EXAMPLE A photo app stores every uploaded photo as an object inside a bucket named my-photo-app. The object's key is users/101/photo1.jpg. In one line: bucket = container, object = file inside it.

Amazon Simple Storage Service (S3) stores files, called objects, inside containers called buckets. It is designed for 99.999999999% (eleven nines) durability, a single object can be up to 5 TB, and you pay for the storage and requests you use.

- **Bucket:** a named container with a globally unique name, created in one Region.
- **Object:** a file plus metadata, identified by a key such as images/logo.png.
- **Not a disk:** S3 is accessed over HTTPS or the API and is not mounted like a drive. Use EBS or EFS when a server needs a file system.

Storage classes

Class	Best for	Retrieval
S3 Standard	Frequently accessed data, websites, analytics	Instant
S3 Intelligent-Tiering	Unknown or changing access patterns	Instant, moves data automatically
S3 Standard-IA / One Zone-IA	Infrequent access, backups	Instant, with a retrieval fee
S3 Glacier Instant Retrieval	Archives read about once a quarter	Milliseconds
S3 Glacier Flexible Retrieval	Long-term archives	Minutes to hours
S3 Glacier Deep Archive	Compliance archives kept for years, lowest cost	Hours

Key features

- **Versioning** keeps older copies so you can recover from deletes or overwrites.
- **Lifecycle rules** move objects to cheaper classes or delete them after a set time.
- **Static website hosting** serves HTML, CSS and JavaScript directly from a bucket.
- **Replication** copies objects to another bucket or Region.
- **Encryption** is applied to new objects by default.
- **Presigned URLs** share a private object for a limited time.

SECURITY Leave Block Public Access on unless you truly host public content. Control access with IAM and bucket policies.

Typical uses: backups, website assets, data lakes, log storage, media files, software downloads.

```
aws s3 mb s3://my-unique-bucket-name --region ap-south-1
aws s3 cp report.pdf s3://my-unique-bucket-name/docs/
aws s3 sync ./site s3://my-unique-bucket-name
aws s3api put-bucket-versioning --bucket my-unique-bucket-name \
 --versioning-configuration Status=Enabled
```

IN DEPTH: Amazon S3

Object storage internals, security layers, cost and performance

How S3 works

- **Flat namespace.** A bucket holds objects addressed by key. Folders in the console are just key prefixes such as logs/2026/10/ that the console draws as folders.
- **Strong consistency.** After a successful write, delete or list, the next read sees the change. You do not need workarounds for eventual consistency.
- **Object parts.** Key, value (the data), version ID, metadata, and optional tags used for access control, billing and lifecycle rules.
- **Durability and availability.** Data is stored redundantly across multiple devices in at least three Availability Zones for most classes. One Zone classes store data in a single AZ and cost less but lose it if that AZ is destroyed.

Uploading large files

Method	Limit / guidance
Single PUT	Up to 5 GB per request.
Multipart upload	Recommended above about 100 MB and required above 5 GB. Up to 10,000 parts of 5 MB to 5 GB each, giving a maximum object of 5 TB. Parts upload in parallel and can be retried individually.
Transfer Acceleration	Routes uploads through the nearest edge location for faster long-distance transfers.
Byte-range GET	Download part of an object, or fetch several ranges in parallel for speed.

COST TRAP Failed multipart uploads leave incomplete parts that you still pay for. Add a lifecycle rule that aborts incomplete multipart uploads after a few days.

Security in layers

Layer	What it does
Block Public Access	Account- or bucket-level switch that overrides any policy or ACL that would make data public. Leave on.
IAM policies	Control what your own users and roles may do.
Bucket policy	Resource policy on the bucket. Allows other accounts, enforces HTTPS, restricts to a VPC endpoint or IP.
Object Ownership	The recommended "bucket owner enforced" setting disables ACLs, so policies are the only access control.
Access Points	Named endpoints with their own policy, handy when many teams share one bucket.
Presigned URLs	Time-limited link to one object, made by someone who has access.
Encryption	SSE-S3 is the default. SSE-KMS adds key control and audit. SSE-C uses keys you supply. Client-side encryption happens before upload.
Object Lock	Write-once-read-many protection (governance or compliance mode) for regulatory retention or ransomware defence.
VPC gateway endpoint	Lets private subnets reach S3 without a NAT gateway or the public internet.

Example: a bucket policy that forces HTTPS

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Sid": "DenyInsecureTransport",
 "Effect": "Deny",
 "Principal": "*",
 "Action": "s3:*",
 "Resource": ["arn:aws:s3:::my-unique-bucket-name",
 "arn:aws:s3:::my-unique-bucket-name/*"],
 "Condition": { "Bool": { "aws:SecureTransport": "false" } }
 }]
}
```

Versioning and delete markers

A bucket is unversioned, versioning-enabled or versioning-suspended. Once enabled it can only be suspended, never removed. Overwriting an object stores a new version. Deleting an object without a version ID adds a **delete marker** that hides the object while older versions remain recoverable. Deleting a specific version is permanent. Old versions cost storage, so pair versioning with a lifecycle rule that expires noncurrent versions.

Lifecycle in practice

Day	Action	Reason
0	Upload to S3 Standard	Hot data, accessed often
30	Transition to Standard-IA	Cheaper once access drops (30-day minimum applies)
90	Transition to Glacier Flexible Retrieval	Archive, retrieval takes minutes to hours
365	Expire (delete)	No longer needed

Watch the fine print: Standard-IA and Glacier Instant Retrieval bill a minimum object size of 128 KB, and the infrequent and archive classes have minimum storage durations (30 days for Standard-IA, 90 days for Glacier Instant and Flexible, 180 days for Deep Archive). Deleting or moving data sooner still charges for the minimum. Many tiny objects can cost more in IA classes than in Standard.

Glacier retrieval options: Flexible Retrieval offers expedited (roughly 1 to 5 minutes), standard (a few hours) and bulk (many hours, cheapest). Deep Archive standard retrieval takes up to about 12 hours and bulk up to about 48 hours.

Replication and events

- **Replication** (same-Region or cross-Region) needs versioning on both buckets and copies only new objects unless you run a Batch Replication job for existing ones. Use it for disaster recovery, compliance copies or lower latency.
- **Event notifications** fire when objects are created or removed and can go to SNS, SQS, Lambda or EventBridge. A typical pattern: image uploaded, Lambda creates a thumbnail, SNS tells subscribers.
- **Static website + CloudFront:** keep the bucket private and let CloudFront read it using Origin Access Control, which adds HTTPS, caching and a custom domain.

Performance and cost

- S3 scales to at least 3,500 write and 5,500 read requests per second per prefix. Spreading data across more prefixes increases total throughput.
- You pay for storage per GB-month, requests, data retrieval for IA and Glacier classes, and data transfer out to the internet. Uploading data in is free, and transfer to EC2 in the same Region is free.
- Use S3 Storage Lens and Cost Explorer to find the buckets and prefixes that cost the most.

Common mistakes

- Making a bucket public to "quickly share a file" instead of using a presigned URL.
- No lifecycle rules, so logs and old versions pile up for years.
- Treating S3 as a file system for frequent small in-place edits.
- Forgetting that bucket names are global and cannot be renamed.

```
aws s3 presign s3://my-unique-bucket-name/docs/report.pdf --expires-in 3600
aws s3api put-bucket-lifecycle-configuration --bucket my-unique-bucket-name \
 --lifecycle-configuration file://lifecycle.json
aws s3api put-public-access-block --bucket my-unique-bucket-name \
 --public-access-block-configuration \
 BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

06 Amazon EC2

Virtual servers you can start in minutes

BASICS: WHAT IS AMAZON EC2 AND WHY USE IT?

What is it?

EC2 (Elastic Compute Cloud) gives you virtual computers, called instances, running in AWS data centres. You choose the operating system and size, start it in minutes, install your software and use it like a computer you rent by the second.

Why is it used?

- Hosting websites, web applications and APIs.
- Running background jobs, data processing and batch work.
- Test and development environments that you can delete after use.
- Software that needs a full server with administrator access.
- Machine learning and graphics work using GPU instances.

Advantages

- Start in minutes with no hardware to buy or maintain.
- Choose the size that fits and change it later.
- Pay per second, so short experiments cost very little.
- Many instance types for different needs (CPU, memory, storage, GPU).
- Full control of the operating system and software.
- Works with Auto Scaling and load balancers to grow and stay available.

Things to watch

- You are responsible for the OS, patches, security settings and backups.
- A forgotten running instance, an unused Elastic IP or a leftover disk keeps costing money.
- One instance is a single point of failure, so use several across two AZs for important apps.
- Data on instance store disks is lost when the instance stops.

Key terms in simple words

Term	Simple meaning	Everyday comparison
Instance	One virtual server.	One rented computer
AMI	A template with the operating system and software used to create instances.	A ready-made installation disc
Instance type	The size and power, such as t3.micro.	Small, medium or large laptop
EBS volume	A network disk attached to an instance that keeps data after stop.	The computer's hard drive
Security group	A firewall that allows traffic by port and source.	A gate with a guest list
Key pair	Files used to log in securely over SSH.	A lock and its key
Elastic IP	A fixed public IP address.	A permanent phone number
User data	A script that runs when the instance first starts.	Setup steps run on first boot

EXAMPLE A small shop's website runs on one t3.micro instance. During a festival sale the owner moves to a bigger type or adds more instances, and afterwards stops paying for them.

Amazon Elastic Compute Cloud (EC2) gives you virtual servers called instances. You choose the operating system, size and network, and you manage everything inside the server. It is the classic IaaS service.

Building block	What it is
AMI	A template with the OS and software used to launch instances.
Instance type	The hardware size, such as t3.micro (family t3, size micro).
EBS volume	A network-attached block disk in one AZ. Persists after the instance stops.
Instance store	Temporary disk on the host. Data is lost when the instance stops or is terminated.
Key pair	Public and private keys for secure SSH (Linux) or password retrieval (Windows).
Security group	A virtual firewall that allows traffic by port and source.
Elastic IP	A static public IPv4 address you can attach to an instance.

Instance families

Family	Letters	Optimised for	Example use
General purpose	T, M	Balanced CPU, memory, network	Web servers, small databases
Compute optimised	C	High CPU	Batch jobs, gaming servers
Memory optimised	R, X	Large memory	In-memory caches, analytics
Storage optimised	I, D	Fast local disk	Data warehouses, NoSQL
Accelerated computing	P, G	GPUs and accelerators	Machine learning, graphics

Ways to pay

Option	How it works	Good for
On-Demand	Pay per second, no commitment	Learning, unpredictable workloads
Savings Plans / Reserved	Commit to 1 or 3 years for a lower rate	Steady, always-on workloads
Spot	Spare capacity at a deep discount; AWS can reclaim it at short notice	Fault-tolerant, flexible jobs

Launching your first instance

- Open the EC2 console and choose Launch instance.
- Pick an AMI (for example Amazon Linux) and an instance type such as t3.micro.
- Create or select a key pair.
- Set the network and a security group, allowing SSH (port 22) only from your own IP.
- Launch, connect, and terminate the instance when finished to avoid charges.

IN DEPTH: Amazon EC2
Instance lifecycle, storage, networking and day-to-day operation

Instance lifecycle and billing

State	What it means	Billing
pending	Starting up	Not billed yet
running	Ready for use	Billed per second (60-second minimum for Linux)
stopping / stopped	Shut down, can be started again; EBS volumes kept	No instance charge, but EBS storage is still billed
hibernated	RAM saved to disk, resumes where it left off	EBS storage billed
shutting-down / terminated	Deleted for good; root volume is deleted by default	Stops billing

REMEMBER Stop keeps the server but its automatic public IP changes on the next start. Terminate deletes it. AWS also bills for public IPv4 addresses by the hour, so release Elastic IPs you no longer use.

Reading an instance type name

Take **m7g.large**: m is the family (general purpose), 7 is the generation, g marks AWS Graviton (Arm) processors, and large is the size. Other common letters: a for AMD, i for Intel, d for local NVMe disks, n for enhanced networking. Newer generations usually give better price and performance.

- **Burstable (T family):** run at a baseline and earn CPU credits while idle, then spend them in bursts. Good for light or spiky workloads. In unlimited mode you can burst beyond credits and pay extra.
- **Graviton (Arm):** often cheaper for the same performance, but software must support Arm.
- **Right-sizing:** check CloudWatch CPU, memory (needs the agent) and network, then move to a smaller or different family if the instance is mostly idle.

Choosing how to pay

Option	Commitment	Notes
On-Demand	None	Full price, per second. Start here when learning or unsure.
Compute Savings Plan	1 or 3 years of a dollar-per-hour spend	Flexible across instance family, size, Region, and also Fargate and Lambda.
EC2 Instance Savings Plan	1 or 3 years, one family in one Region	Larger discount, less flexible.
Reserved Instances	1 or 3 years	Older model, still available. Convertible type allows changes.
Spot	None	Large discounts. AWS gives a two-minute warning before reclaiming. Use for stateless, retryable work and spread across several instance types.
Dedicated Hosts / Instances	Varies	Physical server for you alone, for licensing or compliance.
Capacity Reservation	None or tied to a plan	Guarantees capacity in an AZ for launches you must not miss.

EBS volumes in depth

Volume type	Use	Notes
gp3 (SSD)	Default for most workloads	Baseline 3,000 IOPS and 125 MiB/s regardless of size; you can raise IOPS and throughput separately.
gp2 (SSD)	Older general purpose	Performance tied to size. gp3 is usually cheaper and better.
io1 / io2 (SSD)	Databases needing consistent high IOPS	You provision the IOPS.
st1 (HDD)	Big sequential reads, logs, data warehouses	Low cost per GB. Cannot be a boot volume.
sc1 (HDD)	Cold, rarely accessed data	Lowest cost HDD. Cannot be a boot volume.

- A volume lives in one AZ and attaches to instances in that AZ. To move it, snapshot it and restore elsewhere.
- **Snapshots** are incremental backups kept in S3-backed storage managed by AWS. You can copy them across Regions and create AMIs from them. Automate them with Data Lifecycle Manager or AWS Backup.
- **Encryption** with KMS covers the volume, its snapshots and data in transit to the instance.
- The root volume is deleted on termination by default. Data volumes are kept unless you set otherwise.

Networking pieces

- **Elastic Network Interface (ENI):** the virtual network card, with a private IP, security groups and a MAC address.
- **Placement groups:** cluster (low latency, one AZ), spread (separate hardware for a few critical instances), partition (large distributed systems).
- **Security groups** are stateful and use allow rules only. Reference another security group as a source instead of an IP, so only your load balancer can reach your servers.

Starting and managing instances

- **User data** is a script that runs as root at first boot, good for installing packages and starting your app.
- **Instance metadata service (IMDSv2)** lets software on the instance read its own ID, region and role credentials at 169.254.169.254, using a session token. Require IMDSv2 for better security.

- **Access:** SSH with a key pair, EC2 Instance Connect, or Systems Manager Session Manager, which needs no open inbound port and logs every session.
- AMIs belong to one Region. Copy an AMI to use it elsewhere. A "golden AMI" with your app pre-installed makes Auto Scaling launches faster and more predictable.

```
#!/bin/bash
# user data example: install and start a simple web server on Amazon Linux
dnf install -y nginx
systemctl enable --now nginx
echo "Hello from $(hostname)" > /usr/share/nginx/html/index.html
```

Health and troubleshooting

EC2 runs two status checks: the **system check** (AWS hardware and network) and the **instance check** (your OS). CloudWatch can alarm on them and automatically recover or reboot an instance.

Symptom	Check first
Cannot SSH in	Security group allows port 22 from your IP; instance has a public IP; subnet route to an internet gateway; key file permissions (chmod 400); correct username (ec2-user, ubuntu).
Website not reachable	Security group allows 80/443; app is listening; NACL allows traffic; target is healthy in the load balancer.
Instance slow	CPU credits exhausted on a T instance; memory or disk full; EBS IOPS limit; wrong instance size.
Unexpected bill	Stopped instances still have EBS; idle Elastic IPs; NAT gateways; large snapshots; data transfer out.

```
aws ec2 create-snapshot --volume-id vol-0123456789abcdef0 --description "before upgrade"
aws ec2 create-image --instance-id i-0123456789abcdef0 --name "web-golden-2026-10" --no-reboot
aws ec2 modify-instance-metadata-options --instance-id i-0123456789abcdef0 \
 --http-tokens required
```

07 Amazon VPC

Your own isolated network inside AWS

BASICS: WHAT IS AMAZON VPC AND WHY USE IT?

What is it?

A VPC (Virtual Private Cloud) is your own private network inside AWS. You choose its IP address range, divide it into subnets and decide exactly what can talk to what, and what can reach the internet.

Why is it used?

- Keep your servers and databases separate from other AWS customers.
- Keep databases and application servers hidden from the public internet.
- Control incoming and outgoing traffic with firewalls and routes.
- Connect AWS to your office or data centre privately.
- Meet security and compliance rules that demand network separation.

Advantages

- Strong isolation and fine control over network traffic.
- Free to create. You pay only for add-ons such as NAT gateways and some endpoints.
- Spread across several Availability Zones for resilience.
- Flexible connections: internet, VPN, Direct Connect, other VPCs.
- Layered security with security groups and network ACLs.

Things to watch

- Wrong routes or firewall rules are the most common cause of "cannot connect" problems.
- NAT gateways bill by the hour and by data processed, so they can surprise you on the bill.
- IP ranges must be planned in advance. Overlapping ranges make connecting networks hard.

Key terms in simple words

Term	Simple meaning	Everyday comparison
CIDR block	The range of IP addresses, written like 10.0.0.0/16.	The house number range of a colony
Subnet	A slice of the VPC inside one Availability Zone.	One lane or block of the colony
Public subnet	A subnet with a route to the internet gateway.	A shop front facing the street
Private subnet	A subnet with no direct route from the internet.	A back office with no street entrance
Route table	Rules that decide where traffic goes.	Road signs
Internet gateway	The door between the VPC and the internet.	The colony's main gate
NAT gateway	Lets private servers go out to the internet but blocks inbound connections.	A reception that places calls for staff
Security group	A firewall on an instance or load balancer.	A guard at each room's door

EXAMPLE A web app puts its load balancer in public subnets, and its servers and database in private subnets. Visitors can reach only the load balancer, never the database directly.

A Virtual Private Cloud (VPC) is your own isolated network where you choose the IP range, subnets and traffic rules. EC2 instances, RDS databases and load balancers all live in a VPC.

Term	Meaning
CIDR block	The IP range of the VPC or a subnet, written like 10.0.0.0/16.
Subnet	A slice of the VPC range inside one AZ. Public if its route table sends traffic to an internet gateway, private if not.

Term	Meaning
Route table	The rules that decide where network traffic from a subnet is sent.
Internet gateway	Connects the VPC to the internet in both directions.
NAT gateway	Lets private resources start outbound connections (for updates) while staying unreachable from the internet. Billed by the hour.
Security group	A stateful firewall on an instance or load balancer. Allow rules only. Return traffic is allowed automatically.
Network ACL	A stateless firewall on a subnet. Supports allow and deny rules.
VPC endpoint	A private path to AWS services such as S3 that does not cross the public internet.
Peering / Transit Gateway	Connects separate VPCs so they can talk privately.

Standard design

- **Public subnets** hold the load balancer and NAT gateway, with a route to the internet gateway.
- **Private subnets** hold application servers and the database, with no direct route from the internet.
- Repeat the layout in a second Availability Zone for resilience.

DEFAULT VPC Every account starts with a default VPC in each Region, with public subnets, so you can launch a test instance at once. Build a custom VPC for real workloads.

IN DEPTH: Amazon VPC
Address planning, routing, firewalls and private connectivity

Planning IP addresses

- Use private ranges: 10.0.0.0/8, 172.16.0.0/12 or 192.168.0.0/16. A VPC IPv4 block can range from /16 (65,536 addresses) to /28 (16).
- AWS reserves 5 addresses in every subnet (the first four and the last). A /24 subnet therefore has 251 usable addresses.
- Plan ranges that do not overlap with other VPCs or your office network, otherwise peering and VPN become painful. You can add secondary CIDR blocks later, but you cannot shrink or resize the original.
- The default VPC uses 172.31.0.0/16.

Example layout for a two-AZ application in 10.0.0.0/16

Subnet	AZ	CIDR	Holds
public-a	ap-south-1a	10.0.0.0/24	Load balancer node, NAT gateway
public-b	ap-south-1b	10.0.1.0/24	Load balancer node, NAT gateway
app-a	ap-south-1a	10.0.10.0/24	EC2 app servers
app-b	ap-south-1b	10.0.11.0/24	EC2 app servers
db-a	ap-south-1a	10.0.20.0/24	RDS primary
db-b	ap-south-1b	10.0.21.0/24	RDS standby

Routing

Every subnet is associated with a route table. Each table has an automatic **local** route so resources in the VPC can always talk to each other. The most specific route wins. What makes a subnet public or private is its routes:

Subnet type	Destination	Target	Effect
Public	10.0.0.0/16	local	Talk inside the VPC
Public	0.0.0.0/0	Internet gateway	Reach the internet and be reachable if the instance has a public IP
Private (app)	10.0.0.0/16	local	Talk inside the VPC

Subnet type	Destination	Target	Effect
Private (app)	0.0.0.0/0	NAT gateway	Outbound only, for updates and APIs
Private (db)	10.0.0.0/16	local	No internet route at all

Security group vs network ACL

	Security group	Network ACL
Applies to	Network interface (instance, load balancer, database)	Whole subnet
State	Stateful: replies are allowed automatically	Stateless: allow return traffic explicitly
Rules	Allow only	Allow and deny
Evaluation	All rules are checked together	Numbered rules, lowest number first, first match wins
Default	No inbound, all outbound for new groups	Default NACL allows everything
Tip	Reference other security groups as sources	Use for broad blocks, such as denying a bad IP range. Allow ephemeral ports 1024-65535 for replies.

A clean pattern: the load balancer security group allows 443 from anywhere; the app security group allows the app port only from the load balancer security group; the database security group allows its port only from the app security group. No server needs a public IP except through the load balancer.

NAT gateways and endpoints

- A **NAT gateway** sits in a public subnet with an Elastic IP. It is created per AZ, so for resilience place one in each AZ and route each private subnet to the NAT in its own AZ. You pay hourly plus per GB processed, which makes it a frequent surprise on the bill.
- **Gateway endpoints** for S3 and DynamoDB are free. They add a route so traffic stays on the AWS network and skips the NAT.
- **Interface endpoints (AWS PrivateLink)** create a private network interface for many other services (such as SQS, SNS, CloudWatch and Systems Manager). They bill hourly plus per GB.
- Use endpoints to remove the internet from the path for sensitive traffic and to cut NAT costs.

Connecting networks

Option	Use it for	Notes
VPC peering	Connect two VPCs	One-to-one and not transitive. CIDRs must not overlap.
Transit Gateway	Hub for many VPCs and on-premises	Scales better than many peerings.
Site-to-Site VPN	Encrypted tunnel to an office or data centre	Runs over the internet. Quick to set up.
Direct Connect	Dedicated private link to AWS	Consistent latency and bandwidth, takes longer to order.
Client VPN	Individual users connecting remotely	Managed OpenVPN-based service.

Seeing and debugging traffic

- **VPC Flow Logs** record accepted and rejected connections (source, destination, port, action) to CloudWatch Logs or S3. They show metadata, not packet contents.
- **Reachability Analyzer** checks the path between two resources and names the component that blocks it.
- **DNS:** enable DNS support and DNS hostnames so instances get resolvable names. Route 53 Resolver handles lookups inside the VPC.

Cannot connect? Check in this order

- Does the instance have a public IP (or Elastic IP) if it needs inbound internet access?
- Does the subnet's route table send 0.0.0.0/0 to an internet gateway?
- Does the security group allow the port and source?
- Does the network ACL allow both directions, including return ports?
- Is the service running and listening on the expected port?

```
aws ec2 create-route-table --vpc-id vpc-0123456789abcdef0
aws ec2 create-route --route-table-id rtb-0123456789abcdef0 \
 --destination-cidr-block 0.0.0.0/0 --gateway-id igw-0123456789abcdef0
aws ec2 associate-route-table --route-table-id rtb-0123456789abcdef0 \
 --subnet-id subnet-0123456789abcdef0
aws ec2 create-vpc-endpoint --vpc-id vpc-0123456789abcdef0 --service-name com.amazonaws.ap-south-1.s3 \
 --route-table-ids rtb-0123456789abcdef0
```

08 Amazon RDS

Managed relational databases

BASICS: WHAT IS AMAZON RDS AND WHY USE IT?

What is it?

RDS (Relational Database Service) is a managed database. AWS installs and runs the database server (MySQL, PostgreSQL, Aurora, SQL Server and others), takes backups, applies patches and handles failover, so you only work with your tables and queries.

Why is it used?

- Apps that store structured data such as customers, orders and payments with relationships between tables.
- Teams that want SQL without spending time on servers, patching and backups.
- Production systems that need automatic backups and high availability.
- Reporting that can be spread across read replicas.

Advantages

- Automated backups with point-in-time restore.
- Automatic patching and monitoring support.
- Multi-AZ gives automatic failover to a standby copy.
- Read replicas for more read capacity.
- Encryption at rest and in transit.
- Easy to resize and pay as you go.

Things to watch

- You cannot log in to the underlying server, so you have less control than a self-managed database.
- Multi-AZ roughly doubles the instance cost, and production databases are not cheap.
- Storage can grow but cannot shrink.
- Writes scale mostly by using a bigger instance, not by adding more of them.

Key terms in simple words

Term	Simple meaning	Everyday comparison
Relational database	Data stored in tables with rows, columns and relationships.	A set of linked Excel sheets
SQL	The language used to read and change data.	The language for asking questions to the database
DB instance	The managed database server.	A rented database machine
Engine	The database software, such as PostgreSQL or MySQL.	The brand of the machine
Multi-AZ	A standby copy in another AZ for failover.	A spare driver ready at another station
Read replica	A read-only copy that shares the read load.	A photocopy people can read
Snapshot	A saved backup you can restore from.	A photograph of the database at a moment
Endpoint	The DNS name your app connects to.	The database's address

EXAMPLE An online store keeps customers, products and orders in RDS PostgreSQL. Multi-AZ keeps it running if one AZ fails, and a read replica serves the sales reports so customers are not slowed down.

Amazon Relational Database Service (RDS) runs a database for you. AWS handles installation, patching, backups and failover. Engines: Amazon Aurora (MySQL and PostgreSQL compatible), MySQL, PostgreSQL, MariaDB, Oracle and Microsoft SQL Server.

Feature	What it does	When to use it
Multi-AZ deployment	Keeps a standby copy in another AZ and fails over automatically	Production, for high availability
Read replicas	Extra read-only copies that spread read traffic	Reporting and read-heavy apps
Automated backups	Daily snapshots plus transaction logs, with point-in-time restore	Always on in production
Manual snapshots	Backups you take and keep until you delete them	Before risky changes
Storage autoscaling	Grows disk space when it runs low	Unpredictable data growth
Encryption	At rest with AWS KMS and in transit with TLS	Sensitive data

REMEMBER Multi-AZ is for availability. Read replicas are for performance. They solve different problems and many production systems use both.

- **Good practice:** place the database in a private subnet and allow connections only from your application's security group.
- **RDS or DynamoDB?** Choose RDS for structured, relational data and SQL. Choose DynamoDB for key-value access at very large scale with no servers to manage.

IN DEPTH: Amazon RDS
High availability, scaling, backups and operations

Building blocks

Piece	What it is
DB instance	The managed database server, with an instance class, storage and an engine.
Endpoint	A DNS name your app connects to. Always use the DNS name, never an IP, because failover changes the IP behind it.
DB subnet group	A set of subnets in at least two AZs where RDS may place the database. Use private subnets.
Parameter group	Engine settings (for example max connections, time zone) that you can change without editing config files.
Option group	Optional engine features, mainly for Oracle and SQL Server.
Security group	Controls which clients can reach the port. Allow only the application's security group.

Instance classes

- **db.t3 / db.t4g** burstable, for development and light workloads.
- **db.m** general purpose, balanced for most production use.
- **db.r / db.x** memory optimised, for large working sets and heavy caching inside the database.
- Graviton-based classes (the g suffix) are often cheaper for the same performance on supported engines.

Storage

- gp3 SSD is the usual choice. io1 / io2 (Provisioned IOPS) suit I/O-heavy production databases.
- Storage autoscaling grows the disk when free space runs low, up to a maximum you set. Storage can grow but it cannot shrink.
- Free space is the thing that most often takes a database down: alarm on FreeStorageSpace.

High availability: Multi-AZ options

Option	How it works	Readable standby?	Failover
Multi-AZ instance	A standby in another AZ receives synchronous replication. On failure the endpoint DNS flips to the standby.	No	Typically one to two minutes
Multi-AZ DB cluster (MySQL, PostgreSQL)	One writer and two readable standbys in three AZs.	Yes, two	Usually much faster
Aurora	Storage is shared and kept as six copies across three AZs. Up to 15 reader instances.	Yes	Often under a minute

Failover is triggered automatically by an AZ outage, instance failure, storage failure, or a maintenance action such as an instance class change. Your application should reconnect on errors, because open connections drop during failover.

Read replicas

- Replication is asynchronous, so a replica can lag behind the primary. Watch the ReplicaLag metric.
- Each replica has its own endpoint. Your application must send read queries there, since RDS does not split traffic for you (Aurora provides a reader endpoint that balances across readers).
- A replica can be promoted to a standalone database, useful for disaster recovery or migrations. Replicas can be cross-Region.
- A replica is for performance and reporting, not a replacement for Multi-AZ.

Backups and recovery

Item	Details
Automated backups	Daily snapshot plus transaction logs. Retention from 1 to 35 days (0 turns them off).
Point-in-time restore	Restore to any second within the retention window. It creates a new instance with a new endpoint.
Manual snapshots	Kept until you delete them. Take one before schema changes and before deleting a database.
Cross-Region copy	Copy snapshots to another Region for disaster recovery.
AWS Backup	Central policy-driven backups across RDS, EBS, DynamoDB and more.

TEST YOUR RESTORES A backup you have never restored is only a hope. Practise a restore into a test instance and time it, so you know your real recovery time.

Security checklist

- Private subnet, not publicly accessible, security group open only to the app tier.
- Encryption at rest is chosen when the database is created. To encrypt an existing unencrypted database, copy a snapshot with encryption on and restore it.
- Require TLS for connections. Store the master password in Secrets Manager (the manage-master-user-password option does this and can rotate it).
- Consider IAM database authentication for short-lived tokens instead of static passwords.
- You have no OS or SSH access to an RDS instance. AWS manages the host, which reduces your patching burden.

Operations

- **Maintenance windows** apply patches. Minor version upgrades can be automatic. Major version upgrades are manual and need testing.
- **Blue/green deployments** create a staging copy of production, let you upgrade or change it, then switch over with little downtime.
- **Monitoring:** CloudWatch metrics (CPUUtilization, FreeableMemory, DatabaseConnections, FreeStorageSpace, ReadIOPS, WriteIOPS, ReplicaLag), Enhanced Monitoring for OS-level detail, and Performance Insights to find slow queries. Export slow-query and error logs to CloudWatch Logs.
- **RDS Proxy** pools connections, which protects the database when many Lambda functions or app instances connect at once.

Scaling and choosing

Need	Approach
More CPU or RAM	Scale up: change the instance class. On Multi-AZ this causes a failover-length interruption, so schedule it.
More read capacity	Add read replicas, add ElastiCache in front, or tune queries and indexes.
More write capacity	Scale up, use Provisioned IOPS, or consider Aurora or a sharded design.
Spiky, unpredictable load	Aurora Serverless v2 adjusts capacity automatically.
Key-value access at huge scale	DynamoDB rather than a relational database.

Cost drivers

- Instance hours (Multi-AZ roughly doubles the instance cost), storage, provisioned IOPS, backup storage beyond your database size, and data transfer.
- Stop or delete test databases. A stopped RDS instance restarts automatically after seven days, so delete it if you are finished.

Common mistakes

- Public access turned on, with a wide-open security group.
- Single-AZ for production, and no tested restore.
- Hitting max connections because every app instance opens many connections. Use pooling or RDS Proxy.
- Running out of disk space with no alarm.

```
aws rds create-db-instance-read-replica --db-instance-identifier mydb-replica \
 --source-db-instance-identifier mydb
aws rds create-db-snapshot --db-instance-identifier mydb --db-snapshot-identifier mydb-pre-change
aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier mydb \
 --target-db-instance-identifier mydb-restored --use-latest-restorable-time
```

09 Elastic Load Balancing

Spread traffic across healthy servers

BASICS: WHAT IS ELASTIC LOAD BALANCING AND WHY USE IT?

What is it?

A load balancer is a traffic distributor. Users send requests to one address, and the load balancer spreads them across several healthy servers, so no single server is overloaded and one failed server does not stop the site.

Why is it used?

- One server cannot handle all the traffic of a busy site.
- To avoid a single point of failure by using servers in two or more AZs.
- To update or replace servers without taking the site down.
- To handle HTTPS in one place instead of on every server.
- To route different URLs to different services.

Advantages

- High availability: unhealthy servers are removed from rotation automatically.
- Scales automatically as traffic grows.
- A single stable DNS name for users.
- Works with Auto Scaling, which registers new servers by itself.
- Can terminate TLS and integrate with AWS WAF for protection.

Things to watch

- It costs by the hour plus usage, so delete idle load balancers.
- A badly designed health check can mark good servers as unhealthy.
- Security groups and subnets must be set correctly or targets will show as unhealthy.

Key terms in simple words

Term	Simple meaning	Everyday comparison
Load balancer	The component that receives traffic and spreads it.	A traffic police officer at a junction
Target	A server, container or IP that receives traffic.	One counter at the bank
Target group	A set of targets with a shared health check.	All counters of one service
Listener	Watches for connections on a port such as 443.	A receptionist at the entrance
Health check	A regular test that a target is working.	A doctor's regular check-up
ALB	Load balancer for web traffic (HTTP and HTTPS), routes by URL.	A smart receptionist who reads the visitor's request
NLB	Very fast load balancer for TCP and UDP traffic.	A high-speed toll gate
Cross-zone	Spreads traffic evenly across all AZs.	Sharing queues across all branches

EXAMPLE A website runs on four EC2 servers in two AZs. Visitors use one web address, the ALB sends each request to a healthy server, and if one server crashes users do not notice.

Elastic Load Balancing (ELB) spreads incoming traffic across several targets (EC2 instances, containers, IP addresses) in one or more AZs and sends requests only to healthy targets.

Type	Works at	Best for
Application Load Balancer (ALB)	Layer 7 (HTTP/HTTPS)	Websites and APIs; routes by URL path, host name or header
Network Load Balancer (NLB)	Layer 4 (TCP/UDP/TLS)	Very high performance, static IPs, non-HTTP protocols

Type	Works at	Best for
Gateway Load Balancer (GWLB)	Layer 3	Inserting firewalls and inspection appliances

Key terms

- **Listener:** checks for connections on a port and protocol, such as HTTPS on 443.
- **Target group:** the set of targets that receive traffic, each group with its own health check.
- **Health check:** a regular probe such as GET /health; failing targets leave rotation.
- **Cross-zone balancing:** spreads traffic evenly across targets in all enabled AZs.
- An ALB can terminate TLS using AWS Certificate Manager certificates.

IN DEPTH: Elastic Load Balancing

Rules, health checks, TLS and troubleshooting

How a load balancer is built

- You choose subnets in at least two AZs, and AWS places a load balancer node in each. The DNS name resolves to those nodes, so always use the DNS name and never a fixed IP (except with NLB static IPs).
- **Scheme:** internet-facing (public subnets, public IPs) or internal (private IPs only, for traffic between tiers).
- **Cross-zone balancing:** always on for ALB. Off by default for NLB, where enabling it adds inter-AZ data charges.

Choosing a type

	ALB	NLB	GWLB
Layer	7 (HTTP, HTTPS, gRPC, WebSocket)	4 (TCP, UDP, TLS)	3, uses GENEVE tunnels
Routing	Host, path, header, query string, method, source IP	Port and protocol only	Sends traffic through appliances
Targets	Instances, IPs, Lambda functions	Instances, IPs, ALB	Virtual appliances
Client IP	Passed in the X-Forwarded-For header	Preserved as the source IP	Preserved
IP addresses	Changing, use DNS name	Static IP or Elastic IP per AZ	Not applicable
Best for	Web apps, APIs, microservices	Extreme performance, static IPs, non-HTTP, PrivateLink	Firewalls and inspection

ALB listener rules

A listener checks a port and protocol. It holds rules evaluated in priority order, each with conditions and an action.

Condition example	Action
Path is /api/*	Forward to the api target group
Host is admin.example.com	Forward to the admin target group
Header X-Mobile is true	Forward to the mobile target group
Protocol is HTTP on port 80	Redirect to HTTPS on port 443
Path is /maintenance	Return a fixed response (for example 503 with a message)
Any request (default rule)	Forward with weights, such as 90 percent to v1 and 10 percent to v2 for a canary release

An ALB can also authenticate users with Amazon Cognito or an OIDC identity provider before traffic reaches your app, and it supports sticky sessions through cookies when an app keeps state on one server.

Health checks

Setting	Meaning	ALB default
Path / port / protocol	What the load balancer requests, such as GET /health	On the traffic port
Interval	Seconds between checks	30
Timeout	How long to wait for a reply	5
Healthy threshold	Successes needed to mark healthy	5
Unhealthy threshold	Failures needed to mark unhealthy	2
Success codes	HTTP codes counted as healthy	200

- Make /health a light endpoint that confirms the app is up. A check that queries the database can make every server fail together if the database slows down, taking the whole site offline.
- If every target in every AZ is unhealthy, the load balancer sends traffic to all of them anyway (fail open) rather than refusing everything.

Timeouts and connection draining

- **Idle timeout** (ALB default 60 seconds) closes connections with no traffic. Raise it for slow or long requests, and keep your server keep-alive longer than the load balancer's.
- **Deregistration delay** (default 300 seconds) lets in-flight requests finish when a target is removed, such as during a deployment or scale-in. Lower it for short requests to speed up deployments.
- **Slow start** ramps traffic to a new target gradually so a cold server is not flooded.

TLS and security

- Terminate HTTPS at the ALB using free certificates from AWS Certificate Manager. SNI lets one listener serve several domains with different certificates.
- Choose a modern security policy to control TLS versions and ciphers. Redirect HTTP to HTTPS.
- Attach AWS WAF to an ALB to filter common web attacks, and put CloudFront in front for caching and DDoS absorption.
- **Security groups:** the ALB group allows 80 and 443 from the internet. The app group allows its port only from the ALB group.

Monitoring and reading errors

- Key CloudWatch metrics: RequestCount, TargetResponseTime, HTTPCode_Target_5XX_Count, HTTPCode_ELB_5XX_Count, HealthyHostCount, UnHealthyHostCount.
- Enable access logs to S3 for per-request detail (client IP, path, latency, status).

Error	Usual meaning	Where to look
502 Bad Gateway	The target closed the connection or sent an invalid response	App crashes, keep-alive timeout shorter than the ALB idle timeout, wrong protocol
503 Service Unavailable	No healthy targets are registered	Target group health, security group blocking the health check, empty target group
504 Gateway Timeout	The target did not answer in time	Slow queries, app overloaded, idle timeout too low, security group or NACL blocking
4xx from the ALB	Bad request or authentication problem	Client request, listener rules, authentication settings

Cost

An ALB bills for each hour it exists plus Load Balancer Capacity Units (LCUs), which measure new connections, active connections, processed bytes and rule evaluations, charged on whichever dimension you use most. Delete idle load balancers.

```
aws elbv2 create-rule --listener-arn <listener-arn> --priority 10 \
 --conditions Field=path-pattern,Values='/api/*' \
 --actions Type=forward,TargetGroupArn=<api-target-group-arn>
aws elbv2 modify-target-group-attributes --target-group-arn <target-group-arn> \
 --attributes Key=deregistration_delay.timeout_seconds,Value=60
aws elbv2 describe-target-health --target-group-arn <target-group-arn>
```

10 EC2 Auto Scaling

Add and remove servers automatically

BASICS: WHAT IS EC2 AUTO SCALING AND WHY USE IT?

What is it?

Auto Scaling adds or removes EC2 servers automatically as demand rises and falls, and replaces servers that fail. You say how many servers you want at minimum and maximum, and the rules for changing them, and AWS keeps the fleet at the right size.

Why is it used?

- Traffic is not constant: busy during the day or sales, quiet at night.
- Adding and removing servers by hand is slow and error-prone.
- To always keep a minimum number of servers running.
- To save money by running fewer servers when demand is low.

Advantages

- Saves cost by paying for extra capacity only when needed.
- Self-healing: failed servers are replaced automatically.
- Handles traffic spikes without manual work.
- Works with a load balancer across several AZs.
- The Auto Scaling service itself has no extra charge.

Things to watch

- A maximum that is too low means the site still slows down during a spike.
- Slow-starting servers delay scaling, so keep the AMI ready and startup short.
- A grace period that is too short can cause servers to be replaced again and again.
- Your application must work when any server can disappear at any time (store data outside the server).

Key terms in simple words

Term	Simple meaning	Everyday comparison
Launch template	Describes what to launch: AMI, type, key pair, security groups.	A recipe for a new server
Auto Scaling group	The fleet of servers managed together.	A team that can grow or shrink
Minimum	The fewest servers that must always run.	The minimum staff on every shift
Desired	How many servers the group is trying to run now.	Staff needed right now
Maximum	The upper limit of servers.	The most staff you can afford
Scaling policy	The rule that adds or removes servers.	A rule: add a counter if the queue is long
Scale out / in	Add servers / remove servers.	Open more counters / close some
Health check	A test used to find and replace failed servers.	A regular fitness test

EXAMPLE A ticket-booking site runs 2 servers normally. When a popular show opens, average CPU rises and Auto Scaling adds servers up to 6. After the rush, extra servers are removed.

Auto Scaling adds or removes EC2 instances as demand changes and replaces unhealthy ones, so you pay for extra capacity only while you need it.

Part	Role
Launch template	Defines what to launch: AMI, instance type, key pair, security groups, user data.

Part	Role
Auto Scaling group (ASG)	The fleet. You set minimum, desired and maximum counts and the subnets (AZs).
Scaling policy	The rule that changes capacity.
Health checks	EC2 status checks, and optionally ELB checks, to detect and replace failed instances.

Scaling policies

Policy	How it works	Example
Target tracking	Keeps a metric near a target value	Hold average CPU at 50%
Step scaling	Adds or removes capacity in steps as an alarm gets worse	+2 instances above 70% CPU, +4 above 85%
Scheduled scaling	Changes capacity at set times	Scale up every weekday at 9:00
Predictive scaling	Uses past patterns to scale ahead of demand	Recurring daily traffic peaks

WITH ELB Attach the ASG to a load balancer's target group. New instances register automatically and receive traffic once healthy. Spread the group across at least two AZs. CloudWatch alarms drive step scaling, and target tracking creates its own alarms.

IN DEPTH: EC2 Auto Scaling

Lifecycle, policies, mixed fleets and safe deployments

The life of an instance in a group

An instance moves through Pending, InService and Terminating before it is Terminated. **Lifecycle hooks** pause an instance at launch or termination so you can run extra steps, such as installing software before it takes traffic or draining connections and uploading logs before it shuts down. **Scale-in protection** stops chosen instances from being terminated.

Launch templates and mixed fleets

- Use launch templates, which are versioned. Launch configurations are the legacy approach and should not be used for new work.
- A **mixed instances policy** lets one group use several instance types and combine On-Demand and Spot capacity, for example a fixed On-Demand base plus Spot for burst. Listing many instance types and choosing a capacity-aware allocation strategy reduces Spot interruptions.
- Put as much as possible into the AMI rather than the user data script. A pre-baked AMI starts serving faster than one that installs everything at boot.

Capacity settings

Setting	Meaning
Minimum	The group never goes below this. Use at least 2 across two AZs for availability.
Desired	What the group is trying to run right now. Scaling policies change this number.
Maximum	A hard ceiling. It protects you from runaway cost and from bugs that scale endlessly.

Health checks

- By default the group replaces an instance if its EC2 status checks fail. Turn on ELB health checks as well so an instance that is running but whose app has failed is also replaced.
- **Health check grace period** (default 300 seconds) gives a new instance time to boot before checks count. Too short and the group will terminate healthy but still starting servers in a loop. Too long and failed servers linger.
- The group also rebalances across AZs, launching in an under-used zone before terminating in an over-used one.

Scaling policies in detail

Policy	How to think about it	Good metric
Target tracking	You state a target and AWS creates and manages the alarms. Easiest and usually best.	ASGAverageCPUUtilization, ALBRequestCountPerTarget, average network in or out
Step scaling	You define alarm thresholds and the capacity change for each step, so large breaches cause large changes.	Any CloudWatch metric
Simple scaling	One alarm, one adjustment, then a cooldown. Older and slower to react.	Avoid for new designs
Scheduled	Changes min, desired or max at set times.	Known events and business hours
Predictive	Learns daily and weekly cycles from past load (needs at least a day of history, looks back up to 14 days) and scales ahead.	Repeating patterns

Worked example: target tracking on CPU

Target is 50% average CPU. The group has 4 instances and CPU rises to 80%. Capacity needed is about 4 x 80 / 50 = 6.4, rounded up to 7. If the maximum is 6, the group scales to 6 and stays capped, which is why a maximum that is too low shows up as high CPU even though scaling is working. Scale-in is deliberately slower and more cautious than scale-out.

- **Instance warm-up** tells the group how long a new instance takes to start contributing, so it does not add yet more capacity based on numbers that have not caught up.
- ALBRequestCountPerTarget is often better than CPU for web apps because it tracks the actual demand per server.
- For queue workers, scale on backlog per instance (queue length divided by running instances) using a custom metric from SQS.

Which instance is terminated on scale-in?

The default termination policy first picks the AZ with the most instances to keep zones balanced, then prefers instances running the oldest launch template or configuration, and breaks ties by closeness to the next billing hour. You can change the policy or protect specific instances.

Deploying new versions safely

- Create a new launch template version with the new AMI.
- Run an **instance refresh**. The group replaces instances gradually while keeping a minimum healthy percentage, so the site stays up.
- Use **warm pools** to keep pre-initialised stopped instances ready, which makes scale-out much faster for slow-starting apps.
- For zero-downtime and easy rollback, blue/green with two groups behind weighted target groups is another option.

Common mistakes

- Minimum of 1 in a single AZ, so one failure means an outage.
- Health check grace period shorter than boot time, causing a launch-terminate loop.
- Only EC2 health checks, so a crashed application on a running server is never replaced.
- No maximum or an unrealistically large one, so a bug or attack scales cost sharply.
- Heavy user data scripts that make new servers slow to become useful.
- Scaling only on CPU for an app that is limited by memory, connections or a downstream database.

Cost: the Auto Scaling group itself is free. You pay for the instances it launches and the CloudWatch metrics and alarms it uses.

> **TIME ZONE** The scheduled action in the example below uses UTC, so 03:30 UTC is 09:00 in India (UTC+05:30).

```
aws autoscaling put-scaling-policy --auto-scaling-group-name web-asg --policy-name cpu50 \
 --policy-type TargetTrackingScaling --target-tracking-configuration '{
 "PredefinedMetricSpecification": {"PredefinedMetricType": "ASGAverageCPUUtilization"},
 "TargetValue": 50.0 }'
aws autoscaling put-scheduled-update-group-action --auto-scaling-group-name web-asg \
 --scheduled-action-name weekday-morning --recurrence "30 3 * * 1-5" --min-size 4
aws autoscaling start-instance-refresh --auto-scaling-group-name web-asg \
 --preferences '{"MinHealthyPercentage": 90, "InstanceWarmup": 120}'
```

11 Amazon CloudWatch

See what your AWS resources are doing, and react automatically

BASICS: WHAT IS AMAZON CLOUDWATCH AND WHY USE IT?

What is it?

CloudWatch is AWS's monitoring service. It collects numbers (metrics) and text records (logs) from AWS services and your applications, shows them as graphs and dashboards, and raises alarms when something goes wrong.

Why is it used?

- To know whether servers, databases and apps are healthy.
- To trigger Auto Scaling or send alerts automatically.
- To search application logs when you debug an error.
- To be told about problems before customers complain.
- To watch costs by setting a billing alarm.

Advantages

- Many AWS metrics appear automatically with no setup.
- Alarms can notify people or trigger actions.
- Logs from many servers are kept in one searchable place.
- Dashboards give a quick view of the whole system.
- Works closely with SNS, Auto Scaling and Lambda.

Things to watch

- Basic metrics are free, but custom metrics, logs, dashboards and alarms are charged.
- Logs are kept forever by default, so set a retention period.
- Memory and disk usage on EC2 need the CloudWatch agent.
- An alarm that is too sensitive creates noise, and one that is too loose misses problems.

Key terms in simple words

Term	Simple meaning	Everyday comparison
Metric	A number measured over time, such as CPU percentage.	The speedometer reading of a car
Namespace	A group of related metrics, such as AWS/EC2.	A drawer for similar items
Dimension	A label that narrows a metric to one resource.	The car's registration number
Alarm	Watches a metric and acts when it crosses a threshold.	A smoke alarm
Log group	A collection of logs of one kind, such as one application.	A file folder of reports
Log stream	Log events from a single source.	One report in the folder
Dashboard	A page of graphs and numbers.	A car dashboard
Agent	Software that sends memory, disk and logs from a server.	A reporter who sends details to the control room

EXAMPLE A CloudWatch alarm watches the average CPU of a web fleet. If it stays above 70 percent for ten minutes, it adds servers and also sends an alert through SNS.

Amazon CloudWatch is AWS's monitoring and observability service. It collects metrics (numbers over time) and logs (text records) from AWS services and your applications, draws dashboards, and fires alarms that can notify you or trigger an action such as scaling. Think of it as the dashboard and alarm system of your cloud.

Why it matters

- You cannot fix what you cannot see: CloudWatch shows CPU, network, errors and latency.
- It is the trigger for automation: alarms drive Auto Scaling and EC2 recovery.
- It centralises logs from servers, Lambda functions and containers in one searchable place.

Core components

Component	What it does	Example
Metrics	Time-ordered data points, grouped into namespaces and identified by dimensions	AWS/EC2 namespace, CPUUtilization, dimension InstanceId
Alarms	Watch one metric (or a math expression) against a threshold and change state	CPU above 70% for 5 minutes
Logs	Log groups hold log streams, each a sequence of events. Set retention per group	/var/log/app.log shipped from EC2
Logs Insights	A query language to search and analyse log data	Count errors per hour
Dashboards	Customisable pages of graphs and numbers, including across Regions	Operations overview of ALB, EC2, RDS
Events / EventBridge	Reacts to changes or runs on a schedule, then triggers a target	Run a Lambda every night
Agent	CloudWatch agent collects memory, disk and custom metrics and logs from servers	Memory use on EC2

Metrics in detail

- **Namespace:** a container for related metrics, such as AWS/EC2, AWS/RDS or AWS/ApplicationELB.
- **Dimension:** a name/value pair that narrows a metric, such as InstanceId or AutoScalingGroupName.
- **Statistic and period:** you view metrics as Average, Sum, Minimum, Maximum or percentiles over a period such as 60 seconds or 5 minutes.
- **Resolution:** many AWS services publish at 1-minute or 5-minute intervals. EC2 basic monitoring is 5 minutes, and detailed monitoring gives 1 minute at extra cost. Custom metrics can use high resolution.
- **Custom metrics:** your application can publish its own numbers, such as orders per minute.

> **IMPORTANT** By default EC2 does not report memory or disk space usage. Install the CloudWatch agent if you need them.

Alarms in detail

An alarm is always in one of three states:

State	Meaning
OK	The metric is within the threshold.
ALARM	The metric has breached the threshold for the number of periods you configured.
INSUFFICIENT_DATA	Not enough data yet, or the metric is not being reported.

- **Threshold and evaluation periods:** for example, average CPU above 70% for 3 consecutive 5-minute periods. This avoids reacting to short spikes.
- **Actions:** send a message to an SNS topic, trigger an Auto Scaling policy, or run an EC2 action such as reboot or recover.
- **Composite alarms:** combine several alarms with AND/OR logic to reduce noise.

Logs in detail

- **Log group:** a collection of streams that share retention and access settings, such as one per application.
- **Log stream:** events from a single source, such as one EC2 instance.
- **Retention:** by default logs are kept indefinitely, so set a retention period to control cost.
- **Metric filters:** turn matching log lines into metrics. For example, count lines containing ERROR, then alarm on that count.
- **Subscriptions and export:** stream logs to Lambda or other services, or export to S3 for long-term storage.

Related services

Service	How it differs from CloudWatch
CloudTrail	Records who did what through API calls (audit). CloudWatch records how resources perform.
AWS Config	Tracks resource configuration changes over time and checks compliance.
X-Ray	Traces requests across microservices to find slow or failing components.

Example: CPU alarm that notifies and scales

```
# Alarm when average CPU of the web-asg group stays above 70% for 10 minutes,
# then publish to an SNS topic
aws cloudwatch put-metric-alarm --alarm-name web-high-cpu \
 --namespace AWS/EC2 --metric-name CPUUtilization \
 --dimensions Name=AutoScalingGroupName,Value=web-asg \
 --statistic Average --period 300 --evaluation-periods 2 --threshold 70 \
 --comparison-operator GreaterThanThreshold \
 --alarm-actions arn:aws:sns:ap-south-1:123456789012:ops-alerts
# Publish a custom metric
aws cloudwatch put-metric-data --namespace MyApp --metric-name OrdersPerMinute --value 42
# Set log retention to 30 days
aws logs put-retention-policy --log-group-name /myapp/web --retention-in-days 30
```

Best practices

- Create a billing alarm before experimenting with any service.
- Alarm on symptoms users feel (error rate, latency), not only on CPU.
- Set a retention period on every log group.
- Send alarm notifications to an SNS topic so the right people are told.
- Use dashboards for the few numbers each team checks daily.

12 Amazon SNS

Send one message to many subscribers

BASICS: WHAT IS AMAZON SNS AND WHY USE IT?

What is it?

SNS (Simple Notification Service) is a messaging service that sends one message to many receivers at the same time. A sender publishes a message to a topic, and everyone subscribed to that topic receives a copy by email, SMS, mobile push, a web address, a queue or a function.

Why is it used?

- Sending alerts, for example when a CloudWatch alarm fires.
- Notifying several systems when one event happens (fan-out).
- Sending email, SMS or mobile notifications to users.
- Keeping parts of an application independent from each other.

Advantages

- Fully managed, with no servers to run, and it scales automatically.
- One publish reaches many subscribers.
- Many delivery types: email, SMS, push, HTTP, SQS, Lambda.
- Message filtering so each subscriber receives only what it needs.
- Retries failed deliveries and supports encryption and access control.

Things to watch

- Standard topics can deliver a message more than once, so receivers should handle duplicates.
- SNS pushes messages and does not store them for later reading. Use SQS when you need a queue.
- Messages are limited to 256 KB.
- SMS and some deliveries cost extra, and email subscribers must confirm first.

Key terms in simple words

Term	Simple meaning	Everyday comparison
Topic	A named channel that receives and spreads messages.	A WhatsApp broadcast list
Publisher	Whoever sends a message to the topic.	The person posting in the group
Subscriber	A receiver of the topic's messages.	A member of the group
Subscription	The link between a topic and a receiver.	Joining the group
Message	The content sent, up to 256 KB.	The post itself
Filter policy	A rule so a subscriber gets only matching messages.	Muting all but certain topics
Fan-out	One message delivered to many targets.	A notice sent to all departments
FIFO topic	A topic that keeps strict order and avoids duplicates.	A numbered queue token system

EXAMPLE When an order is placed, the app publishes one message to an SNS topic. Billing, shipping and email each have their own subscription and receive the message at the same time.

Amazon Simple Notification Service (SNS) is a fully managed publish/subscribe messaging service. A sender publishes a message to a topic, and SNS pushes a copy to every subscriber of that topic: email, SMS, mobile push, HTTP endpoints, SQS queues or Lambda functions. The publisher does not need to know who is listening.

How it works

- **Topic:** a named channel that receives messages and fans them out.
- **Publisher:** anything that sends a message, such as a CloudWatch alarm, your application or an S3 event.
- **Subscription:** links a topic to an endpoint. Email subscribers must confirm before they receive messages.
- **Message attributes and filter policies:** let each subscriber receive only the messages it cares about.

Topic types

Type	Behaviour	Best for
Standard	Very high throughput, best-effort ordering, at-least-once delivery (duplicates are possible)	Alerts, notifications, most fan-out
FIFO	Strict ordering and deduplication, with SQS FIFO queues as subscribers, lower throughput	Financial or order processing where sequence matters

Subscriber (endpoint) types

Endpoint	Typical use
Email / Email-JSON	Alerts to people, such as alarm notifications to the ops team
SMS	Text messages to phone numbers, such as one-time codes or urgent alerts
Mobile push	Push notifications to apps through services such as APNs and Firebase
HTTP / HTTPS	Call a webhook on your own service
Amazon SQS	Queue the message for reliable, independent processing
AWS Lambda	Run code immediately when a message arrives
Kinesis Data Firehose	Archive messages to S3 or analytics destinations

Key features

- **Fan-out:** one publish reaches many targets at the same time.
- **Message filtering:** a subscription filter policy delivers only matching messages, so one topic can serve many purposes.
- **Delivery retries and dead-letter queues:** failed deliveries are retried, and undelivered messages can be sent to an SQS dead-letter queue.
- **Encryption:** server-side encryption with KMS, and TLS in transit.
- **Access control:** topic policies and IAM decide who can publish and subscribe.
- **Message size:** messages are limited to 256 KB. Put larger data in S3 and send a link.

SNS vs SQS vs EventBridge

	SNS	SQS	EventBridge
Model	Publish / subscribe, push	Queue, consumers pull	Event bus with routing rules
Delivery	Pushes to many subscribers at once	Each message processed by one consumer	Routes events to targets by rules
Best for	Notifications and fan-out	Decoupling and absorbing spikes	Reacting to AWS, SaaS and custom events

COMMON PATTERN SNS plus SQS fan-out: publish one order event to an SNS topic, and subscribe separate SQS queues for billing, shipping and email. Each team processes the event at its own pace and a slow consumer cannot block the others.

Example: alerts through SNS

```
# Create a topic and subscribe an email address (the recipient must confirm)
aws sns create-topic --name ops-alerts
aws sns subscribe --topic-arn arn:aws:sns:ap-south-1:123456789012:ops-alerts \
 --protocol email --notification-endpoint you@example.com
# Publish a test message
aws sns publish --topic-arn arn:aws:sns:ap-south-1:123456789012:ops-alerts \
 --subject "Test alert" --message "Hello from SNS"
```

Best practices

- Use one topic per purpose, such as ops-alerts or order-events.
- Use filter policies instead of many near-duplicate topics.
- Make subscribers idempotent, because standard topics can deliver a message more than once.
- Restrict who can publish with a topic policy, and encrypt sensitive topics with KMS.
- Attach a dead-letter queue to important subscriptions.

How CloudWatch and SNS work together

A CloudWatch alarm changes state, publishes to an SNS topic, and SNS emails the team, sends an SMS and invokes a Lambda function that opens a ticket. Monitoring detects the problem, and messaging tells the right people and systems.

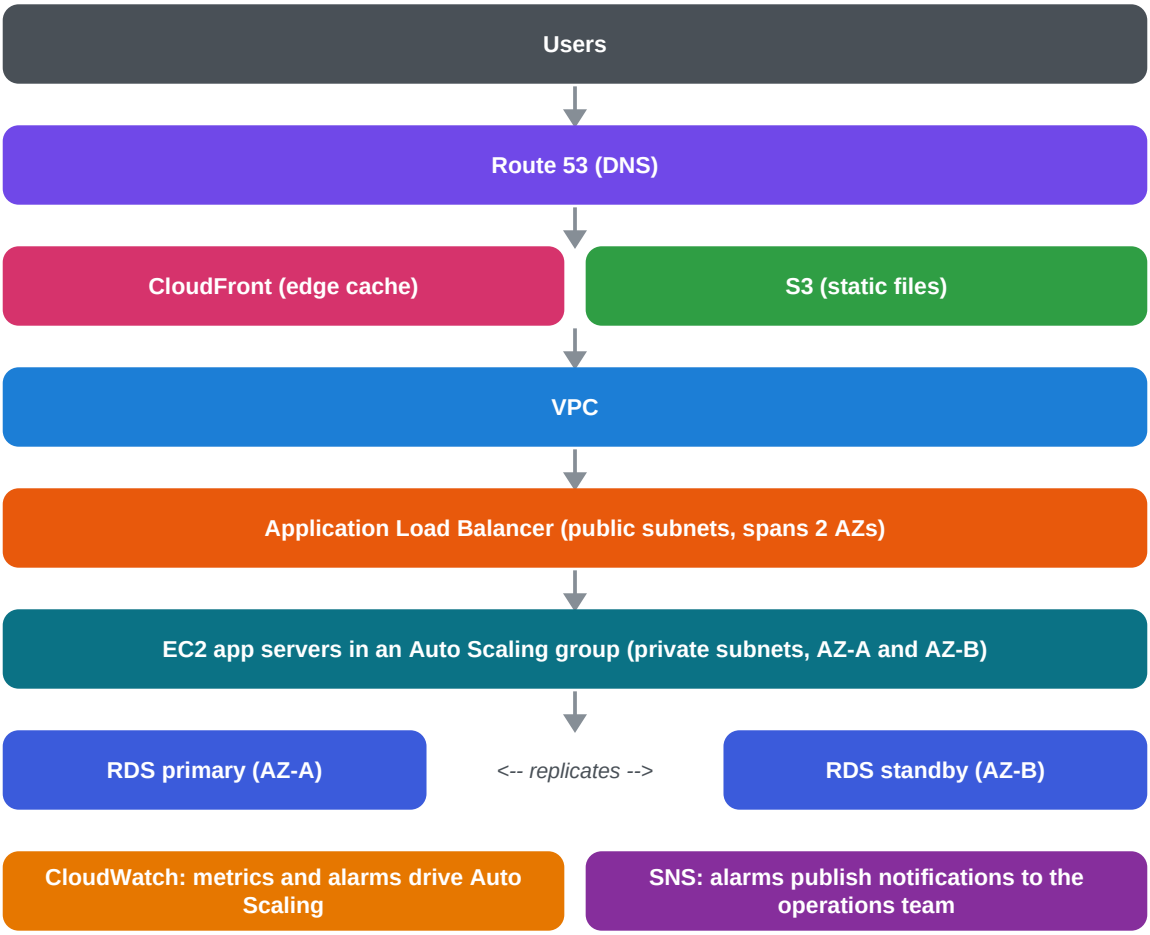
13 Other related services

Companions you will meet next

Service	Category	What it does
Route 53	DNS	Translates domain names to IP addresses; supports health-checked failover and routing policies
CloudFront	Content delivery	Caches content at edge locations for fast responses worldwide
CloudTrail	Auditing	Records who did what in your account through API calls
Lambda	Serverless compute	Runs code in response to events; no servers to manage, pay per run
EBS / EFS	Block / file storage	EBS: a disk for one instance. EFS: a shared file system for many
DynamoDB	NoSQL database	Fully managed key-value and document database with fast, predictable performance
SQS	Messaging queue	Holds messages so parts of an app work independently and absorb spikes
ElastiCache	In-memory cache	Redis, Valkey or Memcached caches that speed up database reads
ECS / EKS	Containers	Run Docker containers; EKS uses Kubernetes
KMS	Security	Creates and manages encryption keys
ACM	Security	Issues and renews TLS certificates for load balancers and CloudFront
CloudFormation	Infrastructure as code	Builds whole environments from a template, repeatably

14 Putting it together

A three-tier web application on AWS



Why this design survives problems

- **A server fails:** the load balancer stops sending it traffic and Auto Scaling launches a replacement.
- **An AZ fails:** the other zone keeps serving and RDS promotes its standby.
- **Traffic spikes:** Auto Scaling adds instances while CloudFront and S3 absorb static requests.
- **You are told:** CloudWatch alarms notify the team through SNS.
- **Security:** only the load balancer is public; servers and database sit in private subnets behind security groups.

15 Best practices and cheat sheet

Build well, spend wisely, keep learning

Well-Architected pillar	Beginner takeaway
Operational excellence	Automate with CloudFormation; monitor with CloudWatch
Security	Least privilege, MFA, encryption, private subnets
Reliability	Use two or more AZs, health checks, backups
Performance efficiency	Right instance and storage type; cache with CloudFront
Cost optimisation	Right-size, stop unused resources, Savings Plans for steady load
Sustainability	Efficient instance types; scale down when idle

Cost-safety checklist

- Create a budget and billing alarm (CloudWatch and SNS can notify you).
- Terminate test EC2 instances and delete unused EBS volumes.
- Release unattached Elastic IPs and delete idle load balancers.
- Delete NAT gateways when practising; they bill by the hour.
- Review the Billing dashboard weekly.

One-page cheat sheet

Service	One-line summary	Remember
IAM	Users, roles and permissions	Least privilege; MFA on root
S3	Object storage in buckets	Not a disk; bucket names are global
EC2	Virtual servers	You manage the OS; terminate to stop charges
EBS	Disk for one EC2 instance	Lives in one AZ
VPC	Your isolated network	Public vs private subnets
RDS	Managed SQL databases	Multi-AZ = availability; replicas = reads
ELB	Spreads traffic across targets	Needs healthy targets in 2+ AZs
Auto Scaling	Adds or removes EC2 instances	Set min, desired and max
CloudWatch	Metrics, logs, alarms, dashboards	Memory and disk need the agent; alarms drive scaling and alerts
SNS	Publish once, deliver to many	Topic + subscriptions; pair with SQS for fan-out
Route 53	DNS	Health-checked failover
CloudFront	Global content cache	Reduces latency and origin load
Lambda	Serverless functions	Pay per execution

16 Interview questions and answers

Common cloud and AWS fundamentals questions, answered briefly

Cloud basics

Q1. What is cloud computing and why do companies use it?

Answer: Cloud computing means renting servers, storage, databases, networking and software over the internet and paying only for what you use. Companies use it for no upfront hardware cost, fast start, elasticity (scale up and down on demand), built-in resilience and global reach. Trade-offs are bills that can grow if unmanaged and dependence on the provider.

Q2. Explain IaaS, PaaS and SaaS with AWS examples.

Answer: IaaS gives raw servers, storage and networks and you manage the OS and above (EC2, EBS, VPC, S3). PaaS means the provider runs servers, OS and runtime and you deploy code or data (Elastic Beanstalk, RDS, Lambda). SaaS is a finished application you just use (Gmail, Microsoft 365, Amazon WorkMail). Moving from IaaS to SaaS you get less control but less work.

Q3. What is the difference between a Region, an Availability Zone and an edge location?

Answer: A Region is a geographic area such as Mumbai (ap-south-1). An Availability Zone is one or more isolated data centres inside a Region with independent power and networking. An edge location is a smaller site that caches content close to users and is used by CloudFront.

Q4. What is the shared responsibility model?

Answer: AWS secures the cloud itself (hardware, facilities). You secure what you put in it: your data, access control and configuration. Analogy: the landlord locks the building and you lock your own flat.

Q5. Is a VPC the same as a private cloud?

Answer: No. A VPC is an isolated network inside the public AWS cloud. It feels private but still runs on shared AWS infrastructure. A true private cloud is used by one organization only, for example on-premises VMware or OpenStack, or AWS Outposts.

IAM

Q6. What is the difference between an IAM user, group and role?

Answer: A user is one person or application with its own long-term credentials. A group is a set of users sharing the same permissions. A role gives temporary permissions that a person or AWS service assumes, with no long-term password, for example an EC2 instance role that lets an app read one S3 bucket.

Q7. How does IAM decide whether a request is allowed?

Answer: Everything is denied by default (implicit deny). An applicable Allow in an identity or resource policy can grant access, but an explicit Deny anywhere always beats any Allow. Guardrails such as SCPs and permissions boundaries only set the maximum and never grant access on their own.

Q8. What are the IAM best practices you follow?

Answer: Enable MFA on the root user and then stop using it, give every person their own identity, grant least privilege, prefer roles over access keys for applications, never commit keys to code, and review unused users and keys using the credential report and Access Advisor.

Amazon S3

Q9. What is S3 and is it a file system?

Answer: S3 is object storage: files (objects, up to 5 TB each) live in buckets with globally unique names and are accessed over HTTPS or the API. It is not a disk for a server. Use EBS (block disk for one instance) or EFS (shared file system) when a server needs a file system.

Q10. Name the S3 storage classes and when you would use each.

Answer: Standard for frequently accessed data. Intelligent-Tiering for unknown or changing access. Standard-IA and One Zone-IA for infrequent access and backups. Glacier Instant Retrieval for archives read about quarterly. Glacier Flexible Retrieval for long-term archives (minutes to hours). Glacier Deep Archive for the cheapest compliance archives (hours).

Q11. How do you share a private S3 object safely without making the bucket public?

Answer: Create a presigned URL, a time-limited link to one object made by someone who has access, for example with aws s3 presign and --expires-in. Keep Block Public Access on.

Q12. What happens when you delete an object in a versioned bucket?

Answer: Deleting without a version ID adds a delete marker that hides the object while older versions stay recoverable. Deleting a specific version is permanent. Versioning can be suspended but not removed, so pair it with a lifecycle rule that expires noncurrent versions to control cost.

Q13. What is a common S3 cost trap?

Answer: Failed multipart uploads leave incomplete parts that you still pay for. Add a lifecycle rule to abort incomplete multipart uploads after a few days. Also remember Standard-IA and Glacier have minimum storage durations and minimum object sizes.

EC2, EBS and VPC

Q14. What is the difference between stopping and terminating an EC2 instance?

Answer: Stopping keeps the instance and its EBS volumes, so you stop paying for the instance but still pay for EBS, and the automatic public IP changes on restart. Terminating deletes the instance for good and the root volume is deleted by default.

Q15. EBS volume versus instance store?

Answer: An EBS volume is a network disk in one AZ that persists after the instance stops and can be snapshotted. Instance store is temporary disk on the host and its data is lost when the instance stops or is terminated.

Q16. What makes a subnet public or private?

Answer: Its route table. A public subnet has a route (0.0.0.0/0) to an internet gateway. A private subnet has no direct route from the internet; it can reach out through a NAT gateway placed in a public subnet.

Q17. Security group versus network ACL?

Answer: A security group is a stateful firewall on the network interface with allow rules only, and replies are allowed automatically. A network ACL is a stateless firewall on the whole subnet with allow and deny rules, evaluated by rule number, so return traffic must be allowed explicitly.

Q18. Why use VPC endpoints?

Answer: They give a private path to AWS services without crossing the public internet. Gateway endpoints for S3 and DynamoDB are free and can cut NAT gateway costs. Interface endpoints (PrivateLink) cover many other services and bill hourly plus per GB.

Q19. You cannot SSH into your EC2 instance. What do you check?

Answer: Security group allows port 22 from your IP, the instance has a public IP, the subnet route table sends 0.0.0.0/0 to an internet gateway, the network ACL allows both directions, key file permissions (chmod 400), and the correct username (ec2-user or ubuntu).

RDS, load balancing and Auto Scaling

Q20. Multi-AZ versus read replicas in RDS?

Answer: Multi-AZ is for availability: a standby in another AZ with synchronous replication and automatic failover. Read replicas are for performance: asynchronous read-only copies that spread read traffic and your app must send reads to their endpoints. Many production systems use both.

Q21. How do you keep an RDS database secure?

Answer: Place it in a private subnet, turn off public access, allow only the app tier security group, encrypt at rest with KMS and require TLS, keep the master password in Secrets Manager, and test restores of automated backups and snapshots.

Q22. ALB versus NLB?

Answer: An ALB works at layer 7 (HTTP/HTTPS) and routes by host, path or header, ideal for websites, APIs and microservices. An NLB works at layer 4 (TCP/UDP/TLS) with very high performance and static IPs per AZ, for non-HTTP protocols.

Q23. What do ALB 502, 503 and 504 errors usually mean?

Answer: 502 Bad Gateway: the target closed the connection or sent an invalid response. 503 Service Unavailable: there are no healthy registered targets. 504 Gateway Timeout: the target did not answer in time, for example slow queries or an idle timeout that is too low.

Q24. Explain minimum, desired and maximum in an Auto Scaling group.

Answer: Minimum is the fewest instances that always run (use at least 2 across two AZs). Desired is what the group is trying to run now and scaling policies change it. Maximum is the hard ceiling that protects against runaway cost.

Q25. Which scaling policies exist and which is usually best?

Answer: Target tracking (hold a metric such as 50% CPU, usually best and easiest), step scaling, scheduled scaling for known events, and predictive scaling for repeating patterns. Simple scaling is older and should be avoided in new designs.

Q26. Why does the health check grace period matter?

Answer: It gives a new instance time to boot before health checks count. If it is shorter than boot time, the group may terminate healthy but still starting servers in a loop. Also enable ELB health checks so a crashed app on a running server is replaced.

CloudWatch, SNS and architecture

Q27. What is CloudWatch and what are its core parts?

Answer: CloudWatch is AWS's monitoring service: metrics (numbers over time), alarms (OK, ALARM, INSUFFICIENT_DATA), logs (groups and streams), Logs Insights, dashboards, EventBridge and the agent. EC2 does not report memory or disk by default, so install the CloudWatch agent for them.

Q28. CloudWatch versus CloudTrail?

Answer: CloudWatch records how resources perform (metrics and logs). CloudTrail is the audit record of who did what through API calls. AWS Config tracks configuration changes over time.

Q29. SNS versus SQS, and what is fan-out?

Answer: SNS is publish/subscribe: it pushes one message to many subscribers (email, SMS, push, HTTP, SQS, Lambda). SQS is a queue where consumers pull and each message is processed by one consumer. In SNS plus SQS fan-out, one event goes to a topic and separate queues for billing, shipping and email process it at their own pace.

Q30. Design a highly available three-tier web app on AWS.

Answer: Route 53 for DNS and CloudFront with S3 for static content. An Application Load Balancer in public subnets across two AZs, EC2 app servers in an Auto Scaling group in private subnets, and RDS Multi-AZ in private subnets. Security groups chain load balancer to app to database, and CloudWatch alarms drive scaling and notify the team through SNS.